

Real World Algorithms

Advanced Graph Algorithms *ITCS 4114: Real World Algorithms*

Kalpathi Subramanian

Computer Science, UNC Charlotte

Last Modified: March, 2026

Learning Outcomes

- ♣ Define a Directed Acyclic Graph (DAG)
- ♣ Apply topological sort to a given application or dataset and generate the topological order
- ♣ Define and explain the greedy approach, as it applies to algorithmic strategies
- ♣ Distinguish the greedy approach between Prim's Minimum Spanning Tree and Dijkstra's shortest path algorithms
- ♣ Apply Prim's Minimum Spanning Tree (MST) algorithm to a given example graph and output the MST
- ♣ Apply Dijkstra's shortest path algorithm to a given example graph and output the shortest path between two vertices

Topological Sort

Problem: Given a set of jobs, courses, etc. with dependencies (constraints), output them in an order that does not violate any of the constraints. Why do we care about this?

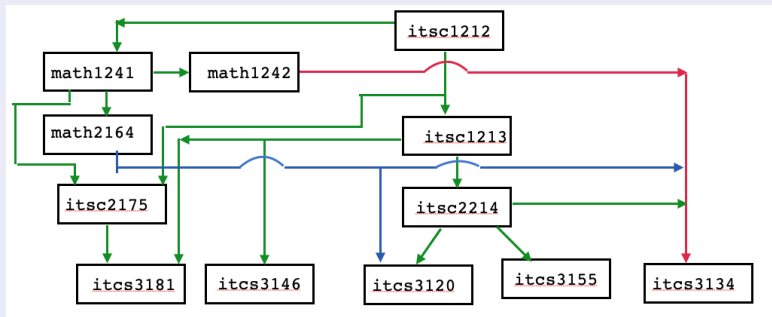
Applications

- ◇ Consider a **large project, like building a skyscraper!**
 - ▷ scheduling multitude of interrelated tasks in proper order is crucial to efficient use of resources.
 - ▷ Need to check the tasks are not contradictory
 - ▷ Overlaps areas or operation research, Critical Path Method, etc.
- ◇ **CS applications:** Instruction scheduling in program compilation, cell evaluation order in spreadsheets, symbol dependencies in program linkers

We are looking at problems that contain a set of constraints and need to be completed in a particular order.

Topological Sort

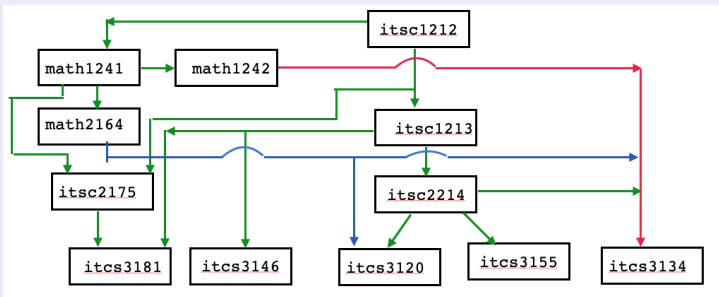
Example: Courses in a Curriculum



- ◇ Model the problem as a DAG - relationships are directed, and acyclic (cycles will cause conflicting constraints).
- ◇ Topological sort can be performed using a **DFS traversal** (reverse order is generated), or using a **source removal** algorithm.

Solving Topological Sort

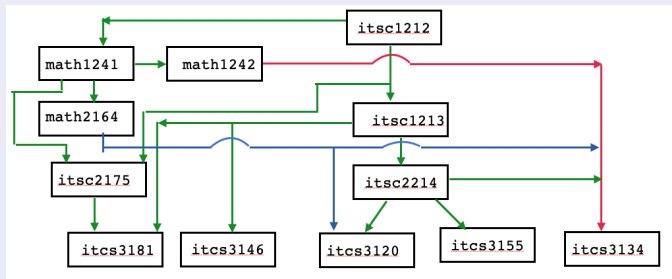
Using DFS



- ◇ Start at any vertex; record the 'pop off' vertices; the reverse order is the topological order.
- ◇ Starting with ITCS2214, the popoff order is:
itcs3120, itcs3155, itcs3134, itsc2214 and stops
- ◇ Restart the DFS from itsc 1212: Popoff order is itcs3181, itsc2175, math2164, math1242, math1241, itcs3146, itsc1213, itsc1212
- ◇ Reverse of the list is the topological order.

Solving Topological Sort

Using Source Removal

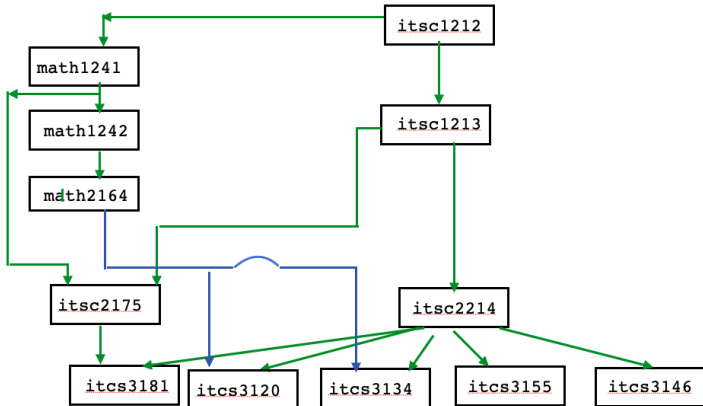


- ◇ Uses the **decrease and conquer** strategy.
- ◇ Repeatedly, identify a source, i.e., a vertex with no incoming edges; remove the vertex and all edges adjacent to it.
- ◇ The order of the removed vertices is the topological order.
- ◇ For our example, the answer can be: itsc1212, itsc1213, itsc2214, itcs3155, itcs3146, math1241, itcs2175, math2164, itcs3120, math1242, itcs3134.

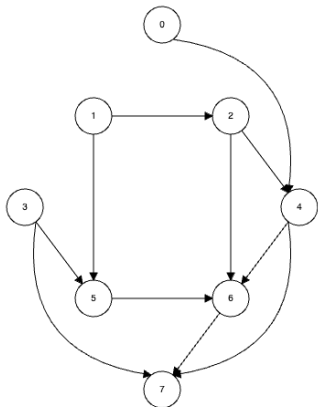
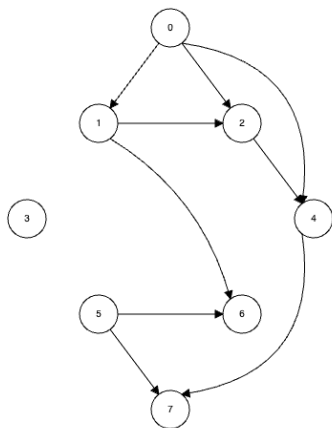
Topological Sort: Using a Queue

```
void toposort(Graph G) {  
    Queue<Integer> Q = new AQueue<Integer>(G.n()); // create a queue  
    int[] Count = new int[G.n()]; // dependency counter  
    int v;  
    for (v=0; v<G.n(); v++) // initialize counters  
        Count[v] = 0;  
    for (v=0; v<G.n(); v++) // update counts by visiting each edge v  
        for (int w = G.first(v); w < G.n(); w = G.next(v, w))  
            Count[w]++;  
    for (v=0; v<G.n(); v++) // find a vertex with no incoming edge  
        if (Count[v] == 0)  
            Q.enqueue(v); // insert into queue  
    while (Q.length() > 0) { // Process the vertices  
        v = Q.dequeue().intValue();  
        printout(v); // output vertex  
        for (int w=G.first(v); w < G.n(); w = G.next(v, w)) {  
            Count[w]--; // One less incoming edge  
            if (Count[w] == 0) // This vertex is now free, add to queue  
                Q.enqueue(w);  
        }  
    }  
}
```

Exercise: Sort the following graph in topological order using DFS



Exercise: Sort the following graphs in topological order
(left graph using DFS, right using source removal)



Greedy Algorithm Design

Constructs a solution to an **optimization problem** piece by piece through a sequence of choices that are:

- ◇ feasible (satisfy the problem's constraints)
- ◇ locally optimal (best local choice among all choices at that point)
- ◇ irrevocable (cannot be changed, no backtracking)

For some problems, yields an optimal solution for every instance; For most, does not but can be useful for fast approximations

Do Greedy Algorithms work?

- ◇ Approach is attractive, offering a simple and appealing solution to the problem by considering a few problem instances
- ◇ More difficult is to prove that greedy algorithm results in an optimal solution.

Greedy Approach Example: Change-making Problem:

Given change for a specific amount n , and m denominations, $d_1 > d_2 > \dots > d_m$ find the fewest coins to add upto n

- ◇ Greedy Strategy: Find the largest coin less than n , continue, until you reach n .
- ◇ Can go in a different order to get the same result, but in this case greedy is optimal solution to the problem.

Given 48 cents, coin denominations of quarters, dimes, pennies, what is the smallest number of coins needed to realize the amount?

- ◇ Choose a quarter first, then two dimes, then 3 pennies
- ◇ Choosing a quarter first, you are *locally optimizing* the problem
- ◇ But it might not always be optimal!
- ◇ Recurrence:

$$F(n) = \min_{j: n \geq d_j} F(n - d_j) + 1, n > 0$$
$$F(0) = 0$$

Greedy Algorithm Design

Remember the line, “**Greed is Good**”?

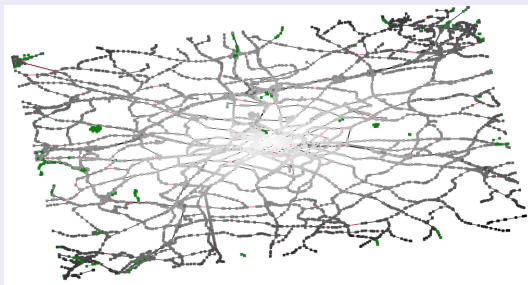
Greedy Algorithm Design: Advantages

- ◇ Greedy algorithms are intuitively appealing and simple
- ◇ Straightforward on how to proceed, given the problem's constraints
- ◇ Proving optimality:
 - ▷ By induction - go from a partially constructed solution to an optimal solution
 - ▷ Moves are as optimal as any other algorithm: moving a knight (chess piece) from lower left corner to the upper right corner.
 - ▷ Show the final result is optimal (placing chips on an 8x8 board so no two chips are adjacent - vertically, horizontally, diagonally)

Shortest Path Problems

Optimization Problems on weighted graphs

- ◇ Problems are non-trivial since the shortest path can be **via a long path of edges of differing weights, rather than the straight edge from source to destination.**
- ◇ Example Application: Road maps (cities and distances) - OpenStreet Map example of Minneapolis (below), complicated by traffic, accidents, construction and other obstacles
- ◇ Other Applications: Packet routing in networks, social networks, speech recognition, robotics, airline crew scheduling.

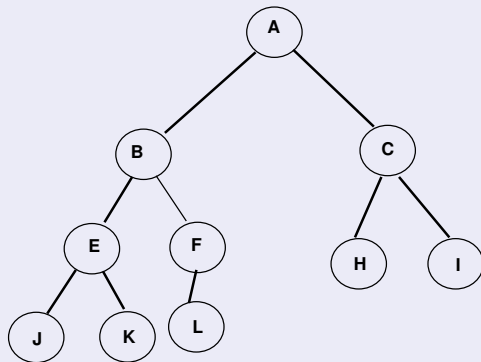


Review: Heaps and Priority Queues

Shortest Path and Minimum Spanning Tree algorithms use Heaps!

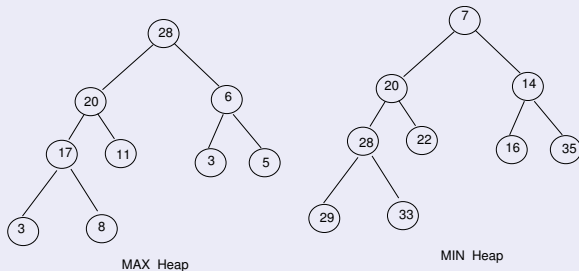
- ◇ Situations where one has to choose the next “most important” from a collection.
- ◇ Examples: patients in an emergency room, scheduling programs in a multi-tasking OS.
- ◇ Need to choose the task with the highest priority
- ◇ A priority queue orders elements by their importance.
- ◇ A queue is insufficient, need an efficient solution.
- ◇ A binary search tree is also insufficient, especially worst case.

A Heap is a Complete Binary Trees



- ◇ Tree of d levels, in which all levels are filled, except possibly the last level
- ◇ Bottom most level has nodes filled in from the left.
- ◇ Advantage: Simple arithmetic to get to children and parents without pointers!

Heap Data Type

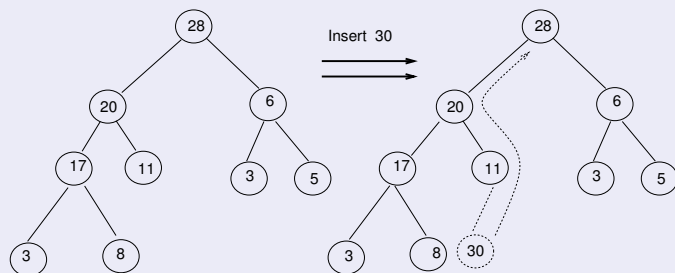


- ◇ Complete binary tree with the **Heap Property**.
- ◇ Heap has **partial ordering**.
- ◇ **Min-heap**: All nodes have values less than child values.
- ◇ **Max-heap**: All nodes have values greater than child values.
- ◇ The root stores the **smallest** value in a min-heap, **largest** value in a max-heap
- ◇ No relationship between node and its sibling in heaps.

Heap Operations

Insert an Element

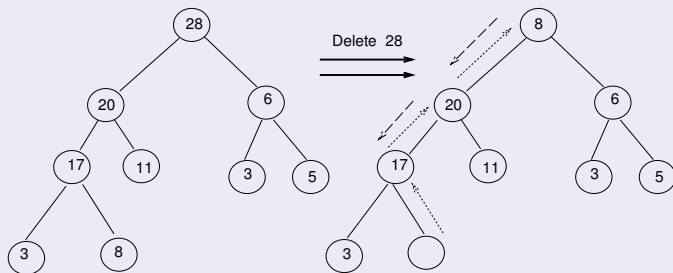
- ◇ Insert element to last position of the array.
- ◇ Reheap using exchanges (move up towards root).
- ◇ Complexity: $O(\log N)$



Heap Operations

Remove Max Element

- ◇ Root element removed.
- ◇ Bring last element to the root.
- ◇ Reheap using exchanges (move down along a path).
- ◇ Complexity: $O(\log N)$



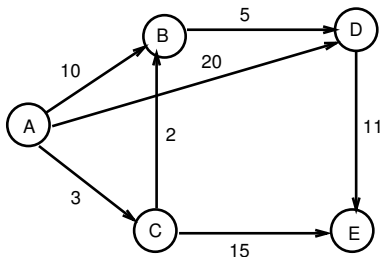
Heap Operations

Remove at position p

- ◇ Exchange element to be deleted with last element.
- ◇ Compare p to its parent and move up as needed, until the root is reached.
- ◇ Sift down from position p .
- ◇ Complexity: $O(\log N)$

Single-Source Shortest Paths

Given a vertex s in graph G , find the shortest path from s to every other vertex in G



- ◇ Closely related problem is to find the shortest (minimum cost) path between two nodes of a graph.
- ◇ This problem is **no more complicated** than the algorithm to find all of the shortest (reachable) paths from a single source.
- ◇ **Bacon number Problem:** Can use BFS on unweighted graphs to find the shortest path (compute the Bacon number).

Degrees of Separation (Bacon Number)

A Graph Application of shortest paths

- ◇ **Problem:** Find degrees of separation of individuals in social networks.
- ◇ **Example:** Actor-Movie network and finding the degree of separation from Kevin Bacon, who is a prolific actor.
- ◇ Bacon number of Kevin Bacon is 0 and all other actors' degree of separation is the number of links away from Kevin Bacon.
- ◇ Goal is to find the minimum number of links (shortest path).
- ◇ Since all the edge weights are the same (assumed to be 1), this reduces to a **BFS traversal**.
- ◇ Goal is to compute the Bacon number and display the path from any actor to Kevin Bacon.
- ◇ **Question:** Why use BFS instead of DFS?

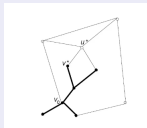
Dijkstra's Shortest Paths Algorithm

Given start vertex s , find the shortest path from s to all other vertices

- ◇ Can visit vertices in some order, $v_0, v_1, \dots, v_{n-1}$,
- ◇ Compute shortest path from v_0 to v_1
- ◇ Next compute shortest path from v_0 to v_2 and compare to path v_0, v_1, v_2 , and so on
- ◇ Algorithm essentially looks at vertices visited so far, then adds shortest path to the next vertex
- ◇ **Problem:** When processing vertex v_i , its shortest path might go through $v_j, j > i$
- ◇ **Solution:** Process vertices in **order of distance from start vertex**, v_s .

Dijkstra's Shortest Paths Algorithm

How does the algorithm work?



- ◇ Algorithm begins from the start vertex and looks at its neighboring (**fringe**) vertices.
- ◇ **Greedy Operation:** Pick the vertex among its adjacent vertices that will result in the shortest path to that vertex. But how?
 - ▷ At iteration i , the algorithm has already computed the shortest paths to i such vertices chosen by the algorithm.
 - ▷ At iteration i , the algorithm chooses the vertex u among the fringe vertices such that the distance of edge vu plus the shortest distance to any of the (tree) vertices.
 - ▷ The chosen vertex is added to the tree vertex list
- ◇ This operation, **that it is sufficient to compare these special paths is the central insight of Dijkstra's algorithm**

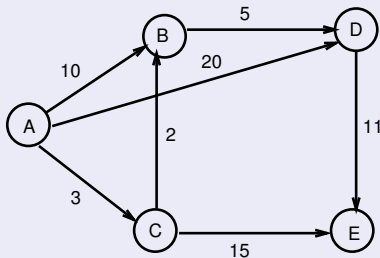
Dijkstra's Algorithm (with Min Heap)

```
DijkstraSSP (G, s) {  
  Initialize (PQ) // priority queue  
  for each vertex  $v \in V$  { // insert vertices into queue  
     $d_v = \infty, p_v = \text{null}$   
    PQ.Insert (PQ,  $v, d_v$ )  
  }  
   $d_s = 0$ ; // Decrease (PQ, s,  $d_s$ )  
   $V_T = \phi$  // tree vertices  
  for (i = 0 to  $|V|-1$ ) {  
     $u^* = \text{DeleteMin(PQ)}$  // remove the smallest distance vertex  
     $V_T = V_T \cup \{u^*\}$  // add to tree vertices  
    for (vertex  $u \in V - V_T$  that is adjacent to  $u^*$  { // fringe vertices  
      if ( $d_{u^*} + w(u^*, u) < d_u$ ) { //updateing fringe vertex distances  
         $d_u = d_{u^*} + w(u^*, u)$   
         $p_u = u^*$   
        Decrease(PQ,  $u, d_u$ ) // can also simply insert into PQ  
      }  
    }  
  }  
}
```

Dijkstra's Algorithm(without Heap)

```
// minDist[v] maintains the smallest distance to a tree vertex v
// V[v] keeps track of the tree edges
void DijkstraSSP(Graph G, int s, HashMap<string,int> minDist,
                 HashMap<string,string> mark) {
    for (string v : G.vertices(); i++) // Initialize distances
        minDist[v] = Integer.MAX_VALUE;
    minDist[s] = 0; // s is the start vertex
    for (int i = 0; i < G.size(); i++) { // Process the vertices
        // scan unvisited vertices and return vertex
        // with smallest distance value
        string v = getMinDistVertex(G, minDist);
        mark[v] = VISITED;
        // update the distances of neighboring vertices of v
        for (string w : G.children(v)) {
            if (minDist[w] > G.EdgeWeight(v, w) + minDist[v]) {
                minDist[w] = G.EdgeWeight(v, w) + minDist[v];
            }
        }
    }
}
```

Dijkstra's Algorithm: Example

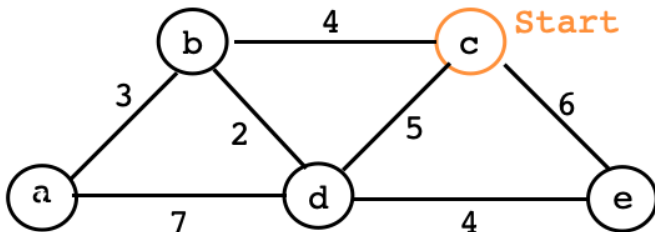
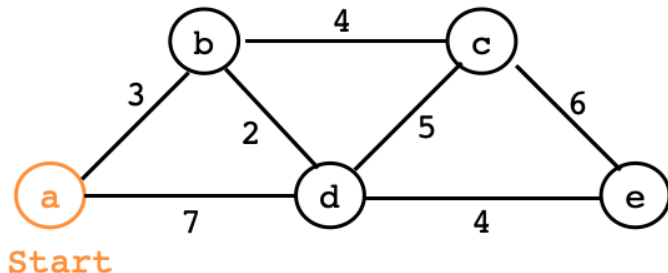


Tree Vertices	Remaining Vertices			
Process A(-, 0)	B(A, 10)	C(A, 3)	D(A, 20)	E(-, ∞)
Process C(A, 3)	B(C, (3+2))	D(A, 20)	E(C, 3+15)	
Process B(C, 5)	D(B, 5+5)	E(C, 18)		
Process D(B, 10)	E(C, 18)			
Process E(C, 18)				

Dijkstra's Algorithm: Analysis

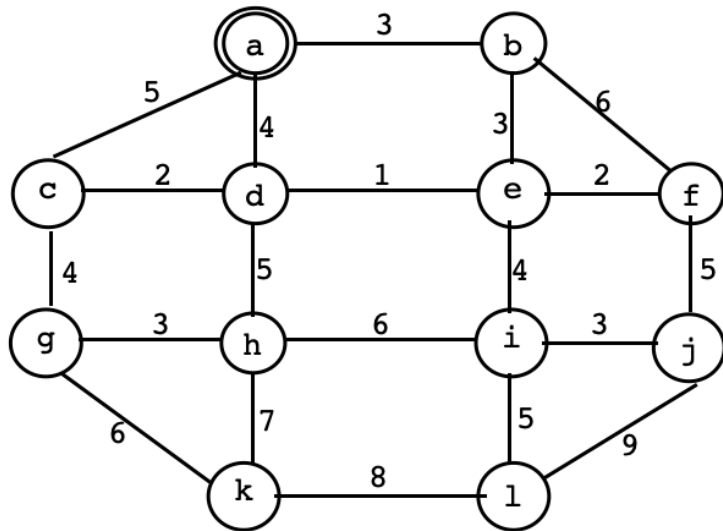
- ◇ Without a Min Heap:
 - ▷ `minVertex()` costs $|V|$ as you have to scan through the vertices and this is done ($|V|$ times, and scanning through all edges, resulting in a complexity $O(|V|^2 + |E|) = O(|V|^2)$, as $|E| \in O(|V|^2)$
- ◇ With Min Heap:
 - ▷ `minVertex()` can be improved using a min-Heap, ordered by D values.
 - ▷ In a minHeap, removing a minimum value, modifications to D values, and reinserting new values, all can be done in $\Theta(\log|V|)$
 - ▷ `minVertex()` takes $\Theta(\log|V|)$, is done $|V|$ times for $|V|\log|V|$
 - ▷ `Decrease()` also takes $\Theta(\log|V|)$ and done $|E|$ times, for $|E|\log|V|$
 - ▷ Total cost: $O(|V| + |E|)\log|V| = |E|\log|V|$
- ◇ For sparse graphs, using a heap is advantageous
- ◇ When using a heap, instead of decreasing values in the heap, can just insert the new record with a smaller value.

Exercise: Apply Dijkstra's Shortest Paths Algorithm on the following graphs, starting at vertices 0 and 4 respectively

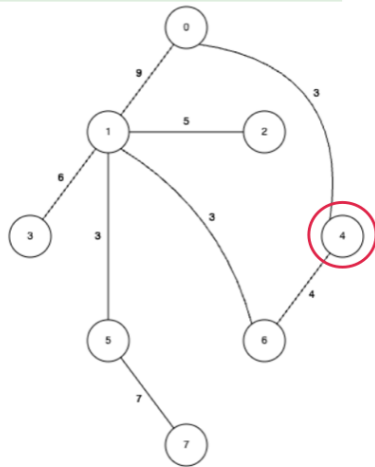
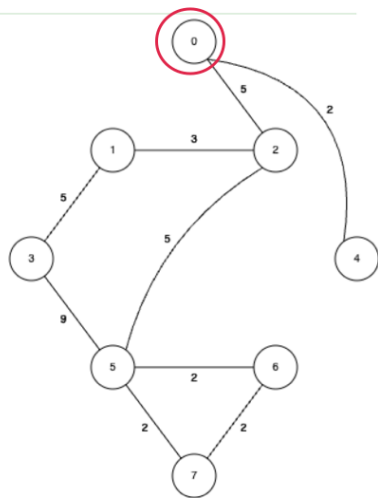


Exercise: Apply Dijkstra's Shortest Paths Algorithm on the following graph, starting at vertex **a**

Start



Dijkstra's Algorithm: Exercise



Minimum Spanning Tree Algorithms

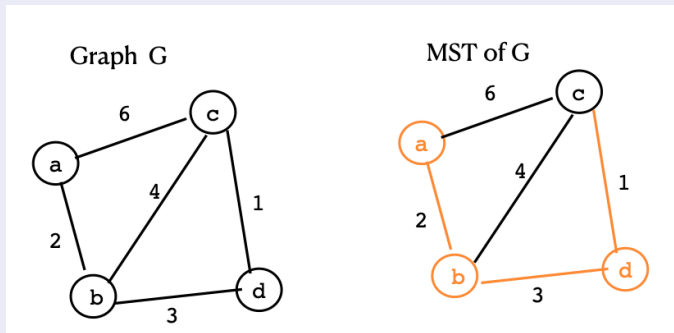
Given n points, connect them in the least expensive way possible such that there is a path between every pair of points

- ◇ Applications in a variety of **Networking** problems
 - ▷ Soldering the shortest set of edges to connect a set of terminals
 - ▷ Connecting cities in a telephone network to minimize cable length.
 - ▷ Transportation, Electrical, Communication networks
 - ▷ **Traveling Salesman problem:** visit a sequence of cities exactly once and return to the starting city at the minimum cost (an instance of the Hamiltonian Cycle).
 - ▷ Applications range across archeology, biology, sociology
- ◇ We use a **graph** representation for the problem, with **edge weights** representing **costs**

Spanning Trees

Definitions

- ◇ **Spanning Tree** of a connected graph G is a **connected acyclic subgraph of G** that includes all of G 's vertices
- ◇ **Minimum Spanning Tree** of a weighted, connected graph G is a spanning tree of G of minimum total weight.

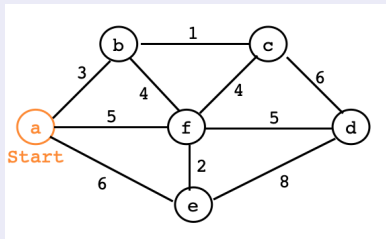


Minimum Spanning Tree: Prim's Algorithm

Approach

- ◇ Builds the minimum spanning tree through a sequence of expanding subtrees
- ◇ Start at any vertex
- ◇ **Greedy Operation:** Each iteration picks a vertex from the unvisited vertices that adds the smallest weight to the MST; adds the edge to the MST
- ◇ When all vertices have been considered, algorithm terminates
- ◇ Output all the edges of the MST

Minimum Spanning Tree: Prim's Algorithm



- ◇ Pick any vertex v to be the start vertex.
- ◇ Start with the MST T_1 , consisting of v ; pick the least cost outgoing edge from v ; if this edge is connected to vertex w , add w and edge vw to the MST.
- ◇ On each iteration, construct the MST, T_{i+1} from T_i by adding a vertex not in T_i , that is closest to those vertices in T_i ("greedy" step).
- ◇ Stop when all vertices are in the tree.

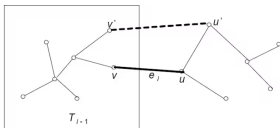
Prim's Algorithm

```
Prim( $G$ ) {  
   $V_T \leftarrow \{v_0\}$   
   $E_T \leftarrow \phi$   
  for  $i \leftarrow 1, |V| - 1$  do  
    find a minimum weight edge  $e^* = (v^*, u^*)$  among all edges  $(v, u)$   
    such that  $v \in V_T$  and  $u \in V - V_T$   
     $V_T \leftarrow V_T \cup \{u^*\}$  // add to tree vertices list  
     $E_T \leftarrow E_T \cup \{e^*\}$  // add to the tree edges list  
  end for  
  return  $E_T$   
}
```

◇ Use a min-Heap to process the min weight edges, for efficiency

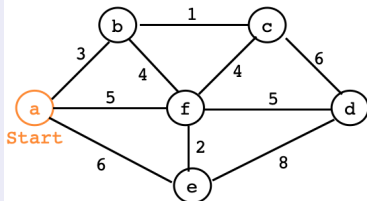
Prim's MST - Correctness Proof

Proof is by contradiction



- ◇ Assume MST has been generated upto a T_{i-1} . Adding a new edge $e(v, u)$ will generate T_i with v in MST and u outside MST.
- ◇ Let T_i not be an MST. Then the added edge will **create a cycle**, since there is some path that goes from v to u .
- ◇ The path will contain some edge $e(v', u')$ with v' in the MST and u' outside MST
- ◇ Thus, Prim's algorithm would have selected $e'(v', u')$, not $e(v, u)$.
- ◇ But Prim's alg. selected the least cost edge, (v, u) , leading to a contradiction.

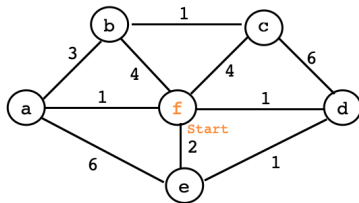
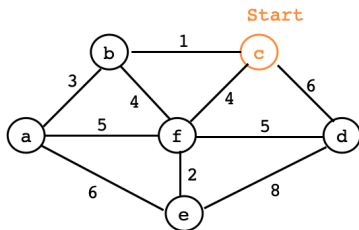
Prim's Algorithm: Example



Tree Vertices	Remaining Vertices				
Choose a(-, -)	b(a, 3)	c(-, ∞)	d(-, ∞)	e(a, 6)	f(a, 5)
Choose b(a, 3)	c(b, 1)	d(-, ∞)	e(a, 6)	f(b, 4)	
Choose c(b, 1)	d(c, 6)	e(a, 6)	f(b, 4)		
Choose f(b, 4)	d(f, 5)	e(f, 2)			
Choose e(f, 2)	d(f, 5)				
Choose d(f, 5)					

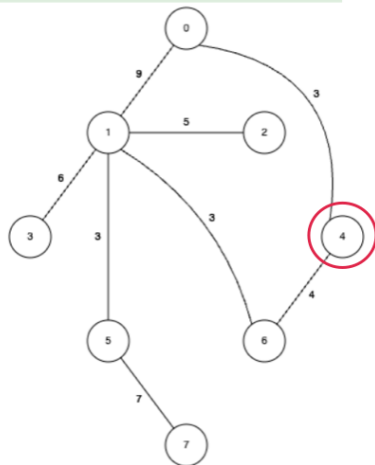
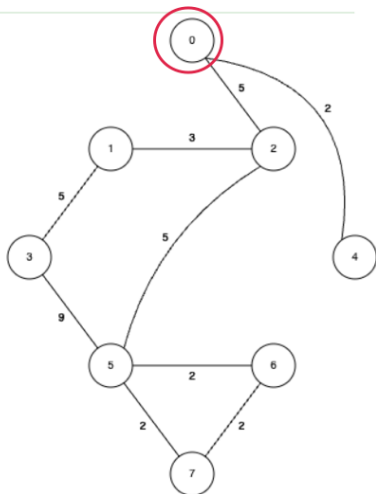
Prim's Algorithm: Exercise

Choose vertex c and f as the starting vertex and run the algorithm



Exercise: Apply Prim's MST algorithm on the following graphs, starting at vertices 0 and 4 respectively

1



Prim's Algorithm: Implementation

Min Vertex by Scanning Graph Vertices

```
void Prim(Graph G, int s, int [] D) {  
    // initialize mark — the list of visited vertices  
    // initialize minDist — distance at each vertex  
    // initialize parent — stores the parent of each MST tree vertex  
    minDist[s] = 0; // s is the start vertex  
    v = getMinVertex (G, minDist)  
    // add edge from parent[v] to v to MST  
    mark[v] = VISITED  
    for (string w : G.children(v)) {  
        if (minDist[w] > weight(w)) {  
            // update minDist  
            // update parent of w  
        }  
    }  
}
```

Prim's Algorithm: Implementation

Min Vertex by Scanning Graph Vertices

```
getMinVertex (Graph G, int [] minDist) {  
    int v = 0  
    for (i: G.vertices) {  
        // find an unvisited vertex, set to v  
    }  
    for (i: G.vertices) {  
        if ((i == UNVISITED) && (minDist[i] < minDist[v]))  
            v = i  
    }  
    return v  
}
```

Prim's Algorithm: Implementation

Min Vertex Using Heap

```
void Prim(Graph G, int s) {  
    // initialize mark — the list of visited vertices  
    // initialize minDist — distance at each vertex  
    // initialize parent — stores the parent of each MST tree vertex  
    // create a heap  
    minDist[s] = 0  
    heap.push ({s, 0}) // push start vertex, its distance into heap  
    for (v : G.vertices) { // only iterate for the number of vertices  
        do { // can have visited vertices in heap!  
            {v,d} = heap.removeElement()  
        } while (mark[v] == VISITED)  
  
        // update the distances of neighboring vertices of v  
        for (string w : G.children(v)) {  
            if (mark[w] == UNVISITED) { // dont have to process visited v  
                if (minDist[w] > G.EdgeWeight(v, w)) {  
                    minDist[w] = G.EdgeWeight(v, w);  
                    parent[w] = v; // needed to output the MST edges  
                    heap.push ({w, G.EdgeWeight(v, w)})  
                }  
            }  
        }  
    }  
}
```

Minimum Spanning Tree: Analysis

◇ Without a Min Heap:

- ▷ `minVertex()` costs $|V|$ as you have to scan through the vertices and this is done ($|V|$ times, and scanning through all edges, resulting in a complexity $O(|V|^2 + |E|) = O(|V|^2)$, as $|E| \in O(|V|^2)$

◇ With Min Heap:

- ▷ `minVertex()` can be improved using a min-Heap, ordered by D values.
- ▷ In a minHeap, removing a minimum value, modifications to D values, and reinserting new values, all can be done in $\Theta(\log|V|)$
- ▷ `minVertex()` takes $\Theta(\log|V|)$, is done $|V|$ times for $|V|\log|V|$
- ▷ `Decrease()` also takes $\Theta(\log|V|)$ and done $|E|$ times, for $|E|\log|V|$
- ▷ Total cost: $O(|V| + |E|)\log|V| = |E|\log|V|$

◇ For sparse graphs, using a heap is advantageous

- ◇ When using a heap, instead of decreasing values in the heap, can just insert the new record with a smaller value.